

UNIVERSITÉ BADJI MOKHTAR - ANNABA

FACULTÉ DES SCIENCES EXACTES

DÉPARTEMENT D'INFORMATIQUE

Rapport de Projet

Classification d'attaques IoT
avec apprentissage supervisé

Réalisé par :

RAMOUL Chouaib

SELOUGHA Achraf

BENAMRANE Achraf

Encadré par :

Dr. BENDJEDDOU

Date de soumission : 4 Janvier 2026

Année universitaire : 2024/2025

Table des matières

Résumé	2
1 Introduction	3
2 Contexte et problématique	3
2.1 Contexte de la sécurité IoT	3
2.2 Problématique	3
3 Méthodologie	4
3.1 Architecture générale	4
3.2 Dataset UNSW-NB15	4
3.3 Distribution des classes	4
4 Implémentation technique	5
4.1 Environnement de développement	5
4.2 Pipeline de prétraitement	5
4.3 Modèle Random Forest	5
5 Résultats et analyse	6
5.1 Performances globales	6
5.2 Matrices de confusion	6
5.2.1 Matrice de confusion absolue	6
5.2.2 Matrice de confusion normalisée	6
5.3 Analyse par classe	7
6 Discussion	7
6.1 Points forts	7
6.2 Limitations	7
6.3 Observations clés	7
7 Conclusion et perspectives	8
7.1 Conclusion	8
7.2 Perspectives	8
7.3 Recommandations	8
A Annexes	8
A.1 Code source principal	8
A.2 Calcul des métriques	9
A.3 Distribution des poids de classes	9
Références bibliographiques	9
Remerciements	10

Résumé

Ce projet aborde le défi crucial de la sécurité dans les réseaux IoT (Internet des Objets) en développant un système intelligent de classification d'attaques. Utilisant le dataset UNSW-NB15, nous avons implémenté un modèle Random Forest avec gestion avancée du déséquilibre des classes. Notre approche combine l'encodage One-Hot des variables catégorielles avec la pondération des classes pour améliorer la détection des attaques minoritaires. Les résultats montrent une précision globale de 70.12% avec des performances remarquables sur les attaques fréquentes (Generic : 96% recall, Reconnaissance : 75% recall). Le rapport présente une analyse détaillée des matrices de confusion et propose des perspectives d'amélioration pour la détection des classes rares.

Mots-clés : IoT, Sécurité réseau, Classification multiclasse, Random Forest, Déséquilibre des classes, UNSW-NB15, Détection d'intrusions.

1 Introduction

L'Internet des Objets (IoT) représente une révolution technologique majeure avec des prévisions de 30 milliards d'appareils connectés d'ici 2025. Cette croissance exponentielle s'accompagne de défis critiques en matière de sécurité, les dispositifs IoT étant particulièrement vulnérables aux attaques en raison de leurs ressources limitées et de leur déploiement à grande échelle.

La diversité des attaques IoT, allant des dénis de service distribué (DDoS) aux exploits sophistiqués et aux malwares spécialisés, nécessite des systèmes de détection adaptatifs et précis. Les approches traditionnelles basées sur des signatures se révèlent insuffisantes face à l'émergence constante de nouvelles menaces.

Dans ce contexte, l'apprentissage automatique supervisé offre une solution prometteuse pour la classification automatique d'attaques. Ce projet vise à développer un système de classification multiclasse capable d'identifier et de catégoriser différents types d'attaques IoT avec une précision élevée.

2 Contexte et problématique

2.1 Contexte de la sécurité IoT

Les réseaux IoT présentent des caractéristiques uniques qui les rendent vulnérables :

- **Ressources limitées** : Mémoire et capacité de calcul restreintes
- **Hétérogénéité** : Diversité des protocoles et architectures
- **Déploiement massif** : Difficulté de mise à jour et de maintenance
- **Connectivité permanente** : Exposition continue aux attaques

2.2 Problématique

Le défi principal réside dans le développement d'un système de classification qui :

1. Identifie précisément 10 types d'attaques différents
2. Gère efficacement le déséquilibre extrême entre classes
3. Maintient des performances en temps réel
4. S'adapte aux nouvelles menaces

3 Méthodologie

3.1 Architecture générale

Notre approche suit le pipeline standard de l'apprentissage automatique :

Données brutes	→	Prétraitement	→	Extraction features
↓				↓
Optimisation	←	Évaluation	←	Entraînement modèle

3.2 Dataset UNSW-NB15

Paramètre	Valeur	Description
Échantillons totaux	257,673	Enregistrements complets
Données d'entraînement	175,341	68% du dataset
Données de test	82,332	32% du dataset
Attributs	45	Features + labels
Classes cibles	10	Types d'attaques + normal

TABLE 1 – Caractéristiques du dataset UNSW-NB15

3.3 Distribution des classes

Le déséquilibre des classes constitue le principal défi :

Classe	Échantillons	Pourcentage
Normal	56,000	31.94%
Generic	40,000	22.81%
Exploits	33,393	19.04%
Fuzzers	18,184	10.37%
DoS	12,264	6.99%
Reconnaissance	10,491	5.98%
Analysis	2,000	1.14%
Backdoor	1,746	1.00%
Shellcode	1,133	0.65%
Worms	130	0.07%

TABLE 2 – Distribution des classes dans le dataset d'entraînement

4 Implémentation technique

4.1 Environnement de développement

- **Plateforme** : Google Colab Pro
- **Langage** : Python 3.9
- **Bibliothèques** : scikit-learn 1.2, pandas 1.5, numpy 1.23

4.2 Pipeline de prétraitement

```
# Encodage des variables catégorielles
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'),
     ['proto', 'service', 'state']),
    ('num', 'passthrough', numerical_cols)
])

# Calcul des poids de classes
class_weights = compute_class_weight(
    'balanced', classes=classes, y=y_train_enc
)
```

4.3 Modèle Random Forest

Configuration optimisée pour le déséquilibre :

- **n_estimators** : 200 arbres
- **class_weight** : Balanced (pondération automatique)
- **max_depth** : Non limité
- **random_state** : 42 (reproductibilité)

5 Résultats et analyse

5.1 Performances globales

Métrique	Baseline	Avec poids	Amélioration
Accuracy	65.09%	70.12%	+5.03%
Macro F1-score	0.35	0.41	+0.06
Weighted F1-score	0.67	0.73	+0.06
Temps d'entraînement	15 min	25 min	+10 min

TABLE 3 – Comparaison des performances des modèles

5.2 Matrices de confusion

5.2.1 Matrice de confusion absolue

RéelPrédit	A	B	D	E	F	G	N	R	S	W
A	11	156	225	201	69	0	15	0	0	0
B	10	66	233	165	74	0	31	2	2	0
D	532	1491	523	887	199	0	387	52	18	0
E	491	1381	519	7434	366	0	784	117	39	1
F	75	221	484	478	2399	0	2322	7	76	0
G	3	16	51	465	45	18161	118	5	7	0
N	865	0	12	691	8922	2	26389	22	97	0
R	67	194	18	352	55	0	150	2634	26	0
S	0	0	2	26	21	0	210	3	116	0
W	0	0	0	37	1	0	4	0	0	2

TABLE 4 – Matrice de confusion absolue (A :Analysis, B :Backdoor, D :DoS, E :Exploits, F :Fuzzers, G :Generic, N :Normal, R :Reconnaissance, S :Shellcode, W :Worms)

5.2.2 Matrice de confusion normalisée

RéelPrédit	A	B	D	E	F	G	N	R	S	W
A	1.62%	23.04%	33.23%	29.69%	10.19%	0.00%	2.22%	0.00%	0.00%	0.00%
B	1.72%	11.32%	39.97%	28.30%	12.69%	0.00%	5.32%	0.34%	0.34%	0.00%
D	13.01%	36.46%	12.79%	21.69%	4.87%	0.00%	9.46%	1.27%	0.44%	0.00%
E	4.41%	12.41%	4.66%	66.78%	3.29%	0.00%	7.04%	1.05%	0.35%	0.01%
F	1.24%	3.65%	7.98%	7.89%	39.57%	0.00%	38.30%	0.12%	1.25%	0.00%
G	0.02%	0.08%	0.27%	2.46%	0.24%	96.24%	0.63%	0.03%	0.04%	0.00%
N	2.34%	0.00%	0.03%	1.87%	24.11%	0.01%	71.32%	0.06%	0.26%	0.00%
R	1.92%	5.55%	0.51%	10.07%	1.57%	0.00%	4.29%	75.34%	0.74%	0.00%
S	0.00%	0.00%	0.53%	6.88%	5.56%	0.00%	55.56%	0.79%	30.69%	0.00%
W	0.00%	0.00%	0.00%	84.09%	2.27%	0.00%	9.09%	0.00%	0.00%	4.55%

TABLE 5 – Matrice de confusion normalisée (recall par classe)

5.3 Analyse par classe

Classe	Précision	Recall	F1-score	Support
Analysis	0.01	0.02	0.01	677
Backdoor	0.02	0.11	0.03	583
DoS	0.25	0.13	0.17	4,089
Exploits	0.69	0.67	0.68	11,132
Fuzzers	0.20	0.40	0.26	6,062
Generic	1.00	0.96	0.98	18,871
Normal	0.87	0.71	0.78	37,000
Reconnaissance	0.93	0.75	0.83	3,496
Shellcode	0.30	0.31	0.31	378
Worms	0.67	0.05	0.09	44
Macro avg	0.43	0.41	0.41	-
Weighted avg	0.78	0.70	0.73	-

TABLE 6 – Performances détaillées par classe d'attaque

6 Discussion

6.1 Points forts

- **Performance globale** : 70.12% d'accuracy avec données déséquilibrées
- **Excellente détection** : Generic (96%), Reconnaissance (75%), Normal (71%)
- **Impact positif** : +5% d'amélioration grâce à la pondération
- **Pipeline robuste** : Reproductible et extensible

6.2 Limitations

- **Classes minoritaires** : Faible performance (Analysis : 2%, Worms : 5%)
- **Confusion inter-classes** : Similarités entre Exploits/DoS, Fuzzers/Normal
- **Ressources** : Temps de calcul élevé (25 minutes)

6.3 Observations clés

1. La pondération améliore significativement le recall des classes minoritaires
2. Les attaques similaires en pattern sont souvent confondues
3. Le déséquilibre extrême nécessite des approches spécialisées
4. La performance est proportionnelle au nombre d'échantillons

7 Conclusion et perspectives

7.1 Conclusion

Ce projet a démontré l'efficacité des méthodes d'apprentissage supervisé pour la classification d'attaques IoT. L'approche Random Forest avec pondération de classes atteint une précision de 70.12% tout en améliorant la détection des attaques minoritaires. Le pipeline développé constitue une base solide pour des systèmes de sécurité IoT.

7.2 Perspectives

1. **Techniques avancées** : SMOTE, ADASYN, Borderline-SMOTE
2. **Algorithmes alternatifs** : XGBoost, LightGBM, réseaux de neurones
3. **Feature engineering** : Sélection automatique, création de features
4. **Déploiement** : Intégration dans système de monitoring réel
5. **Optimisation** : Hyperparameter tuning systématique

7.3 Recommandations

- Prioriser la détection des classes critiques (DoS, Exploits)
- Implémenter un système d'apprentissage continu
- Développer une interface de visualisation des alertes
- Valider sur des datasets IoT spécifiques

A Annexes

A.1 Code source principal

```
# Pipeline complet du modèle
model_weighted = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('classifier', RandomForestClassifier(
        n_estimators=200,
        class_weight=class_weight_dict,
        random_state=42,
        n_jobs=-1
```

```

    ))
])

```

A.2 Calcul des métriques

```

from sklearn.metrics import classification_report
print(classification_report(y_test_enc, y_pred,
    target_names=label_encoder.classes_))

```

A.3 Distribution des poids de classes

Classe	Poids
Analysis	8.77
Backdoor	10.04
DoS	1.43
Exploits	0.53
Fuzzers	0.96
Generic	0.44
Normal	0.31
Reconnaissance	1.67
Shellcode	15.48
Worms	134.88

TABLE 7 – Poids de classes calculés automatiquement

Références bibliographiques

Références

- [1] Moustafa, N., & Slay, J. (2015). *UNSW-NB15 : a comprehensive data set for network intrusion detection systems*.
- [2] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
- [3] Pedregosa, F., et al. (2011). *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- [4] Al-Fuqaha, A., et al. (2015). *Internet of Things : A survey on enabling technologies*. IEEE Communications Surveys & Tutorials, 17(4), 2347-2376.
- [5] He, H., & Garcia, E. A. (2009). *Learning from imbalanced data*. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263-1284.

Remerciements

Nous tenons à exprimer notre sincère gratitude au Dr. BENDJEDDOU pour son encadrement précieux, ses conseils avisés et son soutien constant tout au long de ce projet. Nous remercions également l'Université Badji Mokhtar - Annaba pour les ressources mises à notre disposition.

Fin du rapport